
Objective

This assignment focuses on exploring a machine learning algorithm not covered in class. You will:

- Understand its mechanics—how it works and its key characteristics.
- Implement it in Python, comparing it to a similar algorithm covered in class, highlighting strengths and weaknesses.
- Provide a clear, concise report and presentation explaining your findings.

♦ **Alternative Option:** If you have a strong interest in a specific algorithm outside the provided list, you may propose it for approval before proceeding.

Assignment Deliverables

Each student (or team of up to 3 members) will complete the following:

1. Written Report (3-5 pages, including references and figures) - 75%

1.1 Introduction (15%)

- Provide an overview of the chosen algorithm and explain its significance.
- Discuss why it is relevant to machine learning and how it differs from algorithms covered in class.
- Highlight its applications in real-world scenarios.

1.2 How It Works (20%)

- Explain the mechanics of the algorithm, including its fundamental principles.
- If applicable, include mathematical formulations and key parameters.
- Provide a high-level intuition behind the algorithm's decision-making process.

1.3 Use Cases & Performance (15%)

- Discuss when and where the algorithm works best.
- Identify its advantages and limitations compared to similar techniques.

- Provide real-world examples of its application.

1.4 Code Implementation (15%)

- Include a short Python snippet demonstrating the algorithm in action.
- Clearly explain the key components of the code, including data preprocessing steps, parameter tuning, and output interpretation.
- Ensure the code is well-documented with comments.

1.5 Comparison / Visualization (20%)

- Select a dataset and compare the research algorithm to a related algorithm covered in class (where applicable; for time-series algorithms, you could consider comparing different subsets of data versus comparing different algorithms).
- Provide visualizations (e.g., AUROC for classification, clustering plots, error distributions) to showcase performance differences.
- Explain any data transformations, assumptions, and key insights from the comparison.

2. Presentation (5 minutes, plus 3-5 minutes for questions) - 25%

2.1 Clarity & Organization (10%)

- Present a well-structured, concise explanation of the algorithm.
- Ensure smooth transitions between topics.

2.2 Key Insights from Implementation (10%)

- Highlight key findings from your implementation and comparison.
- Discuss strengths and limitations based on your results.

2.3 Visualizations or Live Demonstration (5%)

- Use relevant visualizations or a brief live code demo to illustrate your findings.
- Ensure visuals are easy to interpret and enhance understanding.

Grading Breakdown

Section	Percentage
Written Report (75%)	
Introduction	10%
How It Works	20%
Use Cases & Performance	15%
Code Implementation	10%
Comparison	20%
Presentation (25%)	
Clarity & Organization	10%
Key Insights from Implementation	10%
Visualizations or Live Demo	5%

Submission Guidelines

- **Report Format:** Submit as a PDF (3-5 pages, including references and figures).
- **Code Submission:** Provide a link to your GitHub repository with the documented code.
- **Presentation:** Be prepared to present in class. All group members must participate.

Topic Sign-Up

- Each topic is available for up to **3 students**.
- Once a topic reaches **3 members**, it will be **closed**.
- A sign-up sheet will be provided—select your topic early to secure your choice!

Final Notes

- Reports should be well-structured and concise; avoid excessive text.
- Late submissions may incur penalties as per course policy.

- Students may use libraries such as Scikit-learn, TensorFlow, PyTorch, and others for implementation.

Topic Selection

Students must [choose an algorithm](#) from the following list or propose a different one for approval:

Supervised Learning

1. Bayesian Regression – A probabilistic extension of linear regression that incorporates prior knowledge and provides uncertainty estimates for predictions.
2. Perceptron Algorithm – A simple binary classification algorithm that updates weights iteratively based on misclassified examples, forming the foundation of neural networks.

Unsupervised Learning

3. Gaussian Mixture Models (GMMs) – A probabilistic clustering method that models data as a mixture of multiple Gaussian distributions, allowing for soft clustering.
4. Independent Component Analysis (ICA) – A technique used for separating independent signals from mixed data, commonly applied in signal processing and feature extraction.

Ensemble Learning (Boosting Methods)

5. AdaBoost – A boosting technique that combines weak classifiers (e.g., decision stumps) iteratively, giving more weight to misclassified points to improve accuracy.
6. XGBoost – An optimized gradient boosting algorithm that improves speed and performance through parallelization and regularization.

Deep Learning & Neural Networks

7. Multi-Layer Perceptron (MLP) – A feedforward neural network with multiple hidden layers that can learn complex patterns in data, commonly used for classification and regression tasks.

Probabilistic & Graph-Based Methods

8. Naïve Bayes – A simple yet effective probabilistic classifier based on Bayes' theorem, assuming independence between features, commonly used in text classification.
9. Markov Chains – A stochastic model used to predict future states based on transition probabilities, widely applied in sequential data modeling.

Meta-Learning & Few-Shot Learning

10. k-Nearest Neighbors with Dynamic Time Warping (DTW-KNN) – A distance-based classification method that uses DTW to compare time series data, making it robust to temporal variations.

Students may propose an alternative topic with instructor approval.